

1 - Python Basics

December 12, 2025

1 1 - Python Basics and Jupyter Notebooks

1.1 Basic Syntax and Types

Some of the primitive types in Python are: - `int` - integer - `float` - floating point number - `str` - string - `bool` - boolean

```
[1]: x = 42 # Variable declaration and assignment
     y = "42" # Creating a variable containing a string
     pi = "3.1415" # Floating point value
     t = True # Create a boolean with value True
```

1.2 Print and f-strings

Use the `print` function to print messages to the standard output.

F-strings (formatted string literals) allow you to format your output and mix constants with values.

To create an f-string, use double quotation marks preceded by an `f`. E.g.: `f"This is a message written with f-string"`.

You can write expressions inside an f-string by using braces `{expression}`. For instance: `f"Variable x has value {x}"`.

```
[2]: ### Printing and f-strings

     print(x) # Use the `print` function to print messages

     print(f"The value of x is {x}. The value of y is {y}")

     # Any expression you write within { } will be evaluated.

     print(f"T = {2*x+10}")
```

42

The value of x is 42. The value of y is 42

T = 94

1.3 Basic operators

Arithmetic: +, -, *, /, //, %, **.

Comparison: ==, !=, <=, >=, <, >. - == - equal - != - not equal - <= - less than or equal to - >= - greater than or equal to - < - less than - > - greater than

Assignment: - = - assignment - += - addition assignment - -= - subtraction assignment - *= - multiplication assignment - /= - division assignment

```
[3]: print("Arithmetic operators")
      print(2 + 3)    # Addition
      print(4 - 10)   # Subtraction
      print(3 * 10)   # Multiplication
      print(10/3)     # Division
      print(10//3)    # Integer division
      print(10%3)     # Modulus (raminder of integer division)
      print(10**3)    # Exponentiation
```

Arithmetic operators

```
5
-6
30
3.3333333333333335
3
1
1000
```

```
[4]: print("Assignment operators")
      a = 10
      print(f"a = {a}")
      a += 5
      print(f"a = {a}")
      a -= 2
      print(f"a = {a}")
      a *= 4
      print(f"a = {a}")
      a /= 2
      print(f"a = {a}")
```

Assignment operators

```
a = 10
a = 15
a = 13
a = 52
a = 26.0
```

1.4 Control flow

The most common control flow structures in Python are: - **if** - conditional - **else** - used after **if** and runs when the condition is not met - **elif** - “else if”, used after an **if** statement - **for** - for loop - **while** - while loop - **break** - loop break - **continue** - loop continue - **pass** - do nothing

A **block** in Python is defined by indentation. These can be spaces or tabs and must be *consistent throughout the entire code*. No mix of spaces and tabs!

PEP 8 style guide: Use spaces only (no tabs!) and 4 spaces at a time. VS Code editor will use four-space blocks by default. It also converts the tab to 4 spaces.

```
[ ]: # For loop with variable i ranging from 0 to 9, but stopping when i == 4

for i in range(10):
    if i == 4: # Four spaces because the if block is inside the for block
        break # Four more spaces (8 in total) because this is inside the if
        ↪block.

# For loop with i ranging from 0 to 19 printing all even numbers
print("Even numbers between 0 and 20:")
for i in range(20):
    if i%2 != 0:
        continue # Skip to next iteration
    print(f" {i}")

# Using a while loop
i = 0
while i < 10:
    i += 1
print(f"While loop ran {i} times.")

# If-else/elif blocks

a = 10
if a > 12:
    print(f"Greater than 12")
elif a > 3:
    print(f"Greater than 3")
else:
    print("3 or less")
```

Even numbers between 0 and 20:

0
2
4
6

```
8
10
12
14
16
18
```

While loop ran 10 times.

1.5 Type casting

In the above example, do `x` and `y` have the same values?

`x` is integer, `y` is a string. We can cast (convert) the string in `y` into an integer with the `int()` function.

```
[5]: # This is false.
print(f"x == y: {x == y}")

# Convert the type of y into integer.
# This is true.
print(f"x == int(y): {x == int(y)}")
```

```
x == y: False
x == int(y): True
```

1.6 Functions

A Python function is defined using the `def` keyword.

```
[ ]: def my_function(a, b): # Define function, note the colon `:`!
      return a - b

print(my_function(10, 20))
```

```
-10
```

1.6.1 Function arguments and keyword arguments

Function arguments can be passed in the order they appear (position arguments), or explicitly named during the function call (keyword arguments).

Arguments may hold **default** values.

```
[8]: # Define a new function with c holding a default value.
def new_function(a, b, c=3):
    return a+b*c

print(new_function(10, 20)) # Position argument call

print(new_function(10, b=3, c=4)) # Keyword argument (assigned by name)
```

```
print(new_function(b=3, c=4, a=5))  # Keyword argument in a different order
print(new_function('d', 'ef'))
```

```
70
22
17
defefef
```

1.7 Data Structures

The basic data structures in Python are: - **list** - array lists, for storing values - **tuple** - immutable sequences, for storing sequences - **dict** - dictionary (map, hash tables), for storing key-value pairs, good - **set** - a hash set, for storing unique values

All of the above data structures are also **iterables**. Which means we can use them within for loops.

```
[ ]: # Create a list using square brackets [ ]
l = [1, 2, 3, 4, 5, 6]
print(f"List: {l}")

# Create a tuple using parentheses ( )
t = (1, 2, 3, 4, 5, 6)
print(f"Tuple: {t}")

# Create a dictionary using braces { }
d = {"pants": 3, "shirts": 6, "shoes": 4}
print(f"Dict: {d}")

# Create a set using braces { }
s = {3, 6, 4, 3}
print(f"Set: {s}")
```

```
List: [1, 2, 3, 4, 5, 6]
Tuple: (1, 2, 3, 4, 5, 6)
Dict: {'pants': 3, 'shirts': 6, 'shoes': 4}
Set: {3, 4, 6}
```

1.7.1 Lists and tuples

Lists and tuples are data structures used to store elements and/or sequences that will not be searched often, and that can be accessed by index.

Lists are mutable, which means you can add and delete elements after the list is created.

Tuples are immutable. Once created, you cannot delete nor add new elements.

Common list methods and functions: - **l.append(x)** - add element **x** to the end of the list - **l.extend(a)** - add multiple elements from a list **a** to the end of the list. - **l.pop(i)** - remove and return element at index **i** from the list. - **l.remove(x)** - removes the first occurrence of **x** from the

list. - `len(l)` - returns the length of the list (that is, the number of elements in the list). - `sum(l)`
- returns the sum of all numbers in `l`. - `sorted(l)` - returns a sorted copy of list `l`.

```
[8]: ### List and tuple basics
```

```
l1 = list(range(10)) # Creates a list with 10 elements from 0 to 9
print(l1)

t1 = tuple(range(10)) # Creates a tuple with 10 elements
print(t1)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
[ ]: ## List operations
```

```
# Add an element to the end of the list
l1.append(10)
print(f"Append 10: {l1}")

# Add multiple elements from another list
l1.extend([11,12,13])
print(f"Extending with [11,12,13]: {l1}")

# Check how many elements are contained in the list
print(f"List l1 has {len(l1)} elements")
```

```
Append 10: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
Extending with [11,12,13]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
```

1.8 Indexing and slicing

Slicing is a technique in Python used to extract segments of a sequence (such as lists, strings, tuples, etc.). It creates new lists from existing ones by providing the starting and ending indices. The starting index is inclusive, the ending index is not, meaning the slice will not contain the element at the last position.

The syntax for slicing is: `sequence[start:end:step]`. Where `start` is the starting index, `end` is the ending index, and `step` is the size of the step (defaults to 1).

```
[1]: l = list(range(20))
```

```
l_slice = l[5:10] # Slice list l from index 5 to 10

print(l_slice)
```

```
[5, 6, 7, 8, 9]
```

1.8.1 Sets

Sets in Python are unordered collections designed to perform efficient membership lookup. Sets do not accept duplicates, therefore, each element in a set is unique. Element duplicates are simply ignored.

Common uses of sets: - Removing duplicates from a list - Perform quick membership lookups - Find common elements between two collections

```
[1]: # Removing duplicates from a list

numbers = [1, 2, 2, 3, 3, 4, 5, 5]
unique_numbers = set(numbers) # Convert list numbers into a set
print(f"List: {numbers}")
print(f"Duplicates removed: {unique_numbers}")
```

```
List: [1, 2, 2, 3, 3, 4, 5, 5]
Duplicates removed: {1, 2, 3, 4, 5}
```

```
[ ]: # Fast membership testing
fruits = {"apple", "banana", "orange", "strawberry"}
print("orange" in fruits) # This should print True
print("grape" in fruits) # This should print False
```

```
True
False
```

```
[3]: # Set operations

a = {1, 2, 3}
b = {3, 4, 5}

print(f"Intersection: {a & b}")
print(f"Union: {a | b}")
print(f"Difference: {a - b}")
```

```
Intersection: {3}
Union: {1, 2, 3, 4, 5}
Difference: {1, 2}
```

1.8.2 Dictionaries

Dictionaries combine the lookup efficiency of sets with the indexing capabilities of lists. Dictionaries store **key-value** pairs. Keys are used as index, it is the parameter you input when looking for something. Values hold the information to be stored/retrieved.

```
[8]: # Creating a dictionary

person = {
    "name": "Alice",
```

```

    "age": 30,
    "city": "Philadelphia"
}

# Access dict fields
print(f"Name: {person['name']}")
print(f"Age: {person['age']}")
print(f"City: {person['city']}")

# Updating fields
person['age'] = 34
print(f"Age updated: {person['age']}")

```

```

Name: Alice
Age: 30
City: Philadelphia
Age updated: 34

```

1.9 Strings

1.9.1 Basic string methods

- `.lower()/.upper()` - convert string to all lower or all upper case.
- `.strip()/rstrip()/lstrip()` - remove whitespaces from the beginning and/or end of the string.
- `.replace(old, new)` - replaces substrings `old` with `new` in a string.
- `.split(sep)` - splits a string into a list using `sep` as the delimiter.
- `.join(iterable)` - joins strings in an iterable.

```

[ ]: ## String Upper/Lower
s = "    Hello World    "
print(s.lower())
print(s.upper())

```

```

hello world
HELLO WORLD

```

```

[ ]: ## String strip
print(s.strip())
print(s.lstrip())
print(s.rstrip())

```

```

Hello World
Hello World
    Hello World

```

```

[ ]: ## String replace

```

```
print(s.replace("World", "Python"))
```

Hello Class

```
[ ]: ## String split
```

```
z = "apple,banana,orange"
a = z.split(",") # Split string z at commas ','.
print(a)
```

['apple', 'banana', 'orange']

```
[ ]: ## Join iterable of strings into a single string
```

```
# Join strings in `a` with a whitespace
print(" ".join(a)) # note: `a` contains the fruits from the above example
```

1.9.2 String search and check

You can search substrings within a string using the following methods:

- `.find(sub)/rfind(sub)` - find the first/last occurrence of a substring. Returns `-1` if not found.
- `.startswith(sub)/.endswith(sub)` - check if the string starts with or ends with substring `sub`.
- `.count(sub)` - count how many times `sub` appears in a substring.

```
[21]: ## Find substrings
```

```
s = "Hello world"
print(s.find("o")) # Find first
print(s.rfind("o")) # Find last
```

4

7

```
[22]: ## Startswith/endswith
```

```
print(s.startswith("He"))
print(s.endswith("rld"))
print(s.startswith("he")) # Case sensitive!
```

True

True

False

1.10 Pythonic idioms

Python has a number of features that permit performing complex computations with few lines of code (or even one-liners).

1.10.1 Ternary operator

The ternary operator in Python performs conditional checks and assign values based on the result of such checks in a single line.

The syntax is: `<True_result> if <True_condition> else <False_result>`.

Consider the program below that checks whether a number `n` is *even* or *odd*.

```
[12]: # Check even or odd

n = 10

if n % 2 == 0:
    result = "even"
else:
    result = "odd"

print(f"n is {result}")
```

n is even

Now, consider the excerpt below that does the same but in one line.

```
[13]: # Checking even or odd with the ternary operator

result = "even" if n % 2 == 0 else "odd"

print(f"n is {result}")
```

n is even

1.10.2 Comprehensions

Python comprehensions are used to quickly create and initialize data structures in a single line of code. We will cover:

- List comprehensions: `[x for x in L]`
- Set comprehensions: `{x for x in L}`
- Dictionary comprehensions `{x: y for x, y in L}`

In the above examples, `L` is an iterable (like a list, set, etc.).

```
[ ]: # Python comprehensions

# Create a list with the squares of numbers from 0 to 9
squares = [i**2 for i in range(10)]
```

```
[ ]: # Using conditionals to filter the above list to include only even numbers
even_squares = [x for x in squares if x % 2 == 0]
```

```
[ ]: # Set comprehensions work similarly to list comprehensions but we use { }
      ↳ instead of [ ]

set_squares = {i**2 for i in range(10)}
```

```
[ ]: # Use list comprehension to flatten a 2D list (matrix)

matrix = [[1,2], [3,4], [5,6]]
flattened = [x for row in matrix for x in row]

print(matrix)
print(flattened)
```

1.11 Zipping, enumerating and unpacking

You can use `zip` to iterate simultaneously over multiple lists (or iterables). *Note:* if both lists are of different sizes, `zip` will discard the remainder of the longest list.

Example:

```
[ ]: ## Create two lists

t1 = list(range(10)) # List numbers from 0 to 9
t2 = list(range(10, 20)) # List numbers from 10 to 19

# Here, the for loop uses two iterators `a` and `b` - one for each list
for a, b in zip(t1, t2):
    print(f"{a} is in t1 | {b} is in t2")
```

```
0 is in t1 | 10 is in t2
1 is in t1 | 11 is in t2
2 is in t1 | 12 is in t2
3 is in t1 | 13 is in t2
4 is in t1 | 14 is in t2
5 is in t1 | 15 is in t2
6 is in t1 | 16 is in t2
7 is in t1 | 17 is in t2
8 is in t1 | 18 is in t2
9 is in t1 | 19 is in t2
```

1.11.1 Lambda functions

Lambda functions are *anonymous* functions defined inline on your program used to evaluate one or more arguments into a single expression.

The syntax is `lambda x: <expression>` where `x` is an input parameter to the function and `<expression>` is any valid Python expression that will produce a return value.

```
[2]: # Defining a lambda function that returns the square of a number

f = lambda x: x**2

print(f(2))
```

4

1.12 The %timeit magic function

Python has a *magic* function for measuring the running time of programs named %timeit.

```
[12]: %%timeit
# Notice the double '%'. This means %timeit will run over the entire program

l = list()

for i in range(100000):
    l.append(i**2)
```

2.99 ms ± 59 s per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
[13]: %%timeit
# Notice the double '%'. This means %timeit will run over the entire program

l = [i**2 for i in range(100000)]
```

2.71 ms ± 21.2 s per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
[1]: # Exercise

# Create a large list and a large set using comprehensions containing numbers
↳ from 0 to 1000000
# Use the membership operator `in` to find the element `999999` in each list
# Measure the running times with %%timeit

large_list = list(range(1000000))
large_set = set(range(1000000))

target = 999999

print("- Measuring search time using `list`")
%timeit target in large_list
```

```
print("- Measuring search time using `set`")
%timeit target in large_set
```

```
- Measuring search time using `list`
4.22 ms ± 70.3 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
- Measuring search time using `set`
15.7 ns ± 0.0429 ns per loop (mean ± std. dev. of 7 runs, 100,000,000 loops
each)
```

1.13 Debugging - Common Python Errors

Python has a number of built-in exceptions (called Errors) that can be used to trace the origin of a problem. Here, we will list some of the most common errors in Python and how to detect them. This procedure is also called **debugging**.

1. **SyntaxError** - occurs when Python cannot parse the code due to incorrect syntax, such as missing parentheses, colons, etc.
2. **NameError** - occurs when a variable or function name is not defined.
3. **TypeError** - occurs when an operation is not valid for an object of that type. For instance, concatenating a string with an integer.
4. **IndexError** - occurs when trying to access an index that does not exist in an iterable.
5. **ValueError** - occurs when a function receives an argument of valid type but inappropriate value.
6. **AttributeError** - occurs when an attribute or method does not exist.
7. **IndentationError** - occurs when the code is not properly indented.

```
[ ]: ## SyntaxError
```

```
def greet(name)
    print("Hello, " + name)
```

```
[ ]: ## NameError
```

```
print(mesage)
```

```
[ ]: ## TypeError
```

```
num = 5
text = "The number is " + num
```

```
[ ]: ## IndexError
```

```
colors = ["red", "green"]
print(colors[2])
```

```
[ ]: ## ValueError
```

```
int("abc")
```

```
[ ]: ## AttributeError
```

```
x = 5  
x.append(3)
```

```
[ ]: ## IndentationError
```

```
def say_hello():  
    print("Hello!")
```